

PROYECTO

Plataforma Distribuida de Telemetría y Gestión de Infraestructura Inteligente

Asignatura: Telemática / Internet: Arquitectura y Protocolos

1. Objetivo

Diseñar, implementar y desplegar una aplicación distribuida que integre programación de redes, sockets TCP y UDP, protocolos de capa de aplicación, concurrencia, servicios web, resolución de nombres, computación en la nube y contenedores.

El proyecto deberá representar una aplicación funcional accesible a través de Internet y permitir que los estudiantes relacionen los conceptos de la arquitectura TCP/IP con el desarrollo de servicios distribuidos reales.

2. Contexto

Una organización encargada de administrar infraestructura inteligente requiere una plataforma para monitorear instalaciones distribuidas geográficamente.

Cada instalación dispone de dispositivos IoT capaces de reportar variables tales como:

- temperatura;
- humedad;
- consumo energético;
- vibración;
- estado operativo de equipos.

La información deberá enviarse a un servidor central desplegado en la nube. Los operadores podrán consultar el estado de los dispositivos y recibir alertas cuando se detecten condiciones anómalas. El proyecto simulará esta infraestructura mediante aplicaciones distribuidas que intercambien información a través de Internet.

3. Arquitectura del sistema

El sistema estará compuesto por tres elementos principales.

3.1. Nodos de telemetría

Representarán dispositivos IoT simulados. Cada nodo deberá:

- tener un identificador único;

- simular 3 o más variables;
- localizar el servidor mediante DNS;
- enviar mediciones periódicamente;
- manejar errores de comunicación.
- Se deberán ejecutar simultáneamente 5 nodos o más.

3.2. Servidor central

Será responsable de:

- recibir y procesar mediciones;
- atender varios clientes simultáneamente;
- identificar los dispositivos conectados;
- detectar valores anómalos;
- generar alertas;
- responder consultas de los operadores;
- manejar desconexiones y errores sin finalizar abruptamente.

El servidor deberá estar desplegado en la nube y ejecutarse dentro de un contenedor Docker.

3.3. Cliente operador

Permitirá:

- consultar nodos activos;
- visualizar las últimas mediciones;
- consultar un dispositivo específico;
- visualizar alertas;
- consultar el estado general del sistema.

El cliente podrá disponer de una interfaz gráfica sencilla.

4. Sockets TCP y UDP

El proyecto deberá implementar comunicación mediante TCP y UDP.

UDP podrá emplearse, por ejemplo, para el envío periódico de telemetría, donde una eventual pérdida de información puede ser tolerable.

TCP deberá utilizarse para operaciones que requieran comunicación confiable, tales como consultas, comandos del operador o intercambio de información crítica.

Los estudiantes deberán justificar en el informe por qué utilizan TCP o UDP para cada servicio.

Ejemplos de mensajes:

TELEMETRY|NODE03|TEMP|24.8

GET_STATUS|NODE03

ALERT|NODE03|TEMP_HIGH|42.1

5. Protocolo de aplicación

Cada equipo deberá diseñar e implementar un **protocolo propio de capa de aplicación basado en texto** para la comunicación principal entre clientes y servidor.

La especificación deberá incluir:

- formato de los mensajes;
- tipos de mensajes;
- parámetros;
- respuestas;
- códigos de error;
- ejemplos de intercambio.

No se permitirá resolver toda la comunicación mediante HTTP, MQTT u otro protocolo de aplicación existente.

6. Programación y concurrencia

El proyecto utilizará **2** lenguajes de programación diferentes, seleccionados entre C, Python, Java, C++, Go u otros autorizados por el docente. **El servidor debe ser en lenguaje C.**

El código deberá evidenciar directamente operaciones relacionadas con sockets:

- creación;
- conexión;
- envío y recepción;
- escucha y aceptación de clientes;

- cierre;
- manejo de excepciones o errores.

El servidor deberá atender concurrentemente los nodos de telemetría y al menos 2 operadores. Para ello podrán emplearse hilos, procesos, multiplexación de sockets u otra estrategia debidamente justificada.

7. Servicio Web

El sistema deberá proporcionar una interfaz web básica para consultar:

- estado del servidor;
- cantidad de nodos registrados;
- nodos activos;
- últimas mediciones;
- alertas recientes.

El propósito no es desarrollar una aplicación web compleja, sino integrar un servicio HTTP con la aplicación distribuida.

8. DNS y despliegue en la nube

El código no deberá contener direcciones IP públicas fijas. Los clientes localizarán el servidor mediante un nombre DNS.

El servidor deberá desplegarse en una plataforma de computación en la nube, preferiblemente **AWS**, utilizando una instancia de cómputo como EC2.

El equipo deberá configurar:

- instancia de cómputo;
- puertos y reglas de acceso;
- DNS;
- acceso remoto;
- conectividad desde equipos externos.

Las pruebas realizadas exclusivamente mediante localhost no serán suficientes para demostrar el funcionamiento del proyecto.

9. Docker

El servidor deberá ejecutarse dentro de un contenedor Docker.

El repositorio incluirá el correspondiente:

Dockerfile

y, cuando la arquitectura lo requiera:

docker-compose.yml

Durante las pruebas deberá demostrarse la construcción de la imagen, ejecución del contenedor y acceso al servicio desde Internet.

10. Análisis de tráfico con Wireshark

Cada equipo realizará capturas de tráfico correspondientes a comunicaciones TCP y UDP.

En el informe deberán identificar y explicar:

- direcciones IP;
- puertos;
- protocolo de transporte;
- establecimiento de una conexión TCP;
- datagramas UDP;
- información del protocolo de aplicación transportada en los paquetes.

Se deberá relacionar lo observado con las capas de **aplicación, transporte y red**.

11. Pruebas del sistema

El equipo deberá demostrar:

- operación simultánea de varios nodos;
- comunicación TCP;
- comunicación UDP;
- consultas desde operadores;
- generación de una alerta;
- funcionamiento concurrente;
- acceso mediante DNS;
- funcionamiento desde Internet;
- recuperación ante una desconexión o mensaje incorrecto.

Adicionalmente, deberá realizar una prueba sencilla donde reporte cantidad de mensajes transmitidos, recibidos y, para UDP, mensajes perdidos si los hubiera.

12. Entregables

La entrega estará conformada por los siguientes elementos.

12.1. Código fuente y repositorio

Se entregará un enlace a un repositorio privado que contenga:

- servidor;
- clientes;
- simuladores;
- interfaz;
- Dockerfile;
- archivos de configuración;
- documentación del protocolo;
- README con instrucciones de ejecución y despliegue.

El repositorio deberá conservar el historial real de desarrollo mediante commits. Durante la sustentación se verificará que no existan modificaciones posteriores a la fecha límite.

12.2. Informe técnico original

El informe deberá ser elaborado específicamente para este proyecto e incluir:

1. Portada e integrantes.
2. Introducción y descripción del problema.
3. Diagrama de arquitectura elaborado por el equipo.
4. Diseño y especificación del protocolo.
5. Implementación de sockets TCP y UDP.
6. Concurrencia y programación de clientes y servidor.
7. Despliegue en nube, DNS y Docker.
8. Análisis de tráfico con Wireshark.
9. Pruebas y resultados.

10. Problemas encontrados y soluciones implementadas.

11. Conclusiones.

Las figuras, tablas y diagramas deberán estar numerados y acompañados de una explicación.

12.3. Evidencias individuales

El informe deberá contener evidencias que permitan verificar la participación de cada integrante.

Cada estudiante aportará pantallazos tomados desde su propio computador donde pueda identificarse:

- nombre de usuario;
- hostname del equipo;
- terminal o IDE;
- ejecución de un componente del proyecto;
- conexión con el servidor desplegado en la nube.

Antes de realizar la prueba puede utilizarse, según el sistema operativo:

whoami

hostname

date

o sus comandos equivalentes en Windows.

La misma terminal deberá mostrar posteriormente la ejecución del componente correspondiente.

También deberán incluirse evidencias del despliegue en nube, contenedor Docker, DNS y capturas realizadas con Wireshark.

No se aceptarán capturas genéricas obtenidas de Internet.

12.4. Video de sustentación

Cada equipo entregará un video de **15 a 18** minutos aproximadamente en el que participen todos los integrantes.

El video deberá mostrar:

1. arquitectura del sistema;

2. protocolo desarrollado;
3. servidor desplegado en la nube;
4. Docker en ejecución;
5. varios nodos funcionando simultáneamente;
6. cliente operador;
7. comunicación TCP y UDP;
8. generación de una alerta;
9. análisis breve de una captura de Wireshark;
10. principales resultados.

Cada integrante deberá explicar la parte que desarrolló y demostrar su funcionamiento.

El video deberá mostrar la ejecución real del proyecto; no será suficiente una presentación compuesta únicamente por diapositivas.

13. Sustentación

Durante la sustentación el docente podrá solicitar modificaciones o pruebas en tiempo real, tales como:

- iniciar o detener un nodo;
- cambiar una medición para generar una alerta;
- consultar un dispositivo;
- explicar un mensaje del protocolo;
- identificar código relacionado con sockets;
- reiniciar el contenedor;
- demostrar resolución DNS;
- interpretar un paquete TCP o UDP.

Todos los integrantes deberán comprender la arquitectura general del sistema y estar en capacidad de explicar su participación.

14. Evaluación

Componente	Porcentaje
Programación y uso de sockets TCP/UDP	25 %
Protocolo y arquitectura distribuida	15 %
Concurrencia y funcionamiento del sistema	15 %
Nube, DNS y Docker	15 %
Wireshark y pruebas de red	10 %
Informe, código y evidencias	10 %
Video y sustentación	10 %
Total	100 %

15. Entrega final

Cada equipo deberá entregar:

1. enlace al repositorio privado;
2. informe técnico en PDF;
3. código fuente completo;
4. documentación del protocolo;
5. Dockerfile y archivos de configuración;
6. instrucciones de despliegue;
7. evidencias individuales;
8. enlace al video de sustentación.

El objetivo del proyecto no es únicamente conseguir que diferentes programas intercambien información. Se espera demostrar la integración y comprensión de:

aplicación → sockets → TCP/UDP → direccionamiento → DNS → Internet → nube → Docker → análisis del tráfico.